Understanding ML, Chapter 2: From Representations to Decisions

After this chapter you can explain what a loss decides, why the Bayes rule is the target of every learner, how kNN imitates it and why it dies in high dimension, what ERM promises and does not promise, why cross-entropy trains calibrated probabilities, and why a number computed on data that chose the model is never a performance estimate

Fundamentals of Machine Learning, midterm

2026-11-03

What this chapter is about, in one paragraph

A learner receives numbers, not objects, so somebody first decides how to describe an object and how to encode the description: that is the *representation*, and it bounds everything that follows. If we knew the whole world, that is the joint distribution of inputs and outputs, the best decision at each input would be a one-line rule: pick the action with the smallest average damage, where "damage" is the *loss* we chose. Different losses pick different summaries of the same uncertainty (mean, median, quantile, most probable class, a cost threshold), so "the best prediction" has no meaning until the loss is named. We never know the world; we have a finite sample. The first idea is to imitate the optimal rule locally, by looking at the nearest examples (kNN), which works until the number of features makes "nearby" meaningless. The second idea is to restrict ourselves to a *family* of rules with adjustable parameters and let the sample pick one member by minimising the average loss on the sample (ERM). That average is honest for a rule chosen in advance and flattering for the rule the sample chose, which is why a model's own training score is never evidence of quality, why the training score can only reward more flexibility, and why we keep validation data to choose settings and test data, touched once, to report performance. Throughout, what we *manipulate* is: the representation, the loss, the neighbourhood size k, the family and its hyperparameters, and which subset of the data is allowed to influence which decision.

Contents

1 From objects to numbers: the representation **2**

2 Deciding when the world is known: the Bayes rule **3**

3 From the known distribution to finite data: nearest neighbours **7**

4 Parameterised models: families of rules **11**

5 Learning the parameters from data: ERM and cross-entropy **13**

6 Evaluating a learned model: train, validation, test **17**

7 The learning problem in one picture **20**

1 From objects to numbers: the representation

Input space, feature vector, one-hot encoding

A learning algorithm can only receive a list of d numbers. Two human choices turn an object into such a list: **describe** it (select which properties to record: area, rooms, age, neighbourhood) and **encode** the description as a vector

$$x = [x_1, \dots, x_d]^\top \in \mathcal{X} \subseteq \mathbb{R}^d \quad \text{[on your sheet]}$$

$\mathcal{X}$ is the **input space**, d the number of **features**, x_j feature j. A numerical property is one component. A categorical property with K possible values becomes K binary components with a single 1 in the position of the value (**one-hot encoding**). A house of 145 m^2, 4 rooms, 12 years, in neighbourhood A of $\{A, B, C\}$ is $x = [145, 4, 12, 1, 0, 0]^\top \in \mathbb{R}^6$.

Purpose. Give the learner the information the decision needs, in a form where "similar inputs" means "similar objects".

Impact. The representation fixes the ceiling: no later stage can recover information that was not encoded, and every distance-based method (kNN, clustering, anything with a scale) inherits whatever geometry the encoding invented.

Three descriptions of the same emails

Setup. One labelled collection of emails (spam or not). Build three representations of each email: (1) its length in characters, $d = 1$; (2) counts of the 10,000 most frequent words, $d = 10{,}000$; (3) the same counts plus 500 columns of random noise, $d = 10{,}500$.

Held fixed. The emails, the labels, the learner (say logistic regression), the split, the metric.

What we manipulate. Only the representation.

What we measure. Validation error of the fitted model for each representation, and how it changes as the training set grows from 500 to 50,000 emails.

What the plot shows. Three curves of error against training size. Curve (1) is flat and high: length cannot separate spam. Curve (2) drops fast. Curve (3) ends at the same level as (2) but needs many more emails to get there.

How to read it. Description (1) lacks information: no amount of data fixes it. Description (3) has the same information as (2) but makes the learner spend data discovering that 500 coordinates are noise. More features are not automatically better; *informative* features are.

Neighbourhoods coded as 1, 2, 3

A real-estate team codes neighbourhoods A, B, C as $1, 2, 3$ to save columns. Their kNN price model then treats a house in A as at distance 1 from a house in B and at distance 2 from one in C: the code invented "B is halfway between A and C". With one-hot coding every pair of neighbourhoods is at the same distance $\sqrt{2}$ (for instance $\|(1,0,0) - (0,1,0)\| = \sqrt{1+1}$), which says "different, and equally so". Same for units: a feature in metres and the same feature in millimetres carry the same information but weigh a thousand times differently in any distance.

Integer codes for categories; forgetting units

People code categories as integers because it is one column instead of K. The learner then reads an order and a spacing that do not exist, so neighbours are chosen for the wrong reason and

the validation error is worse than the data allow. Fix: one-hot encode unless the categories are genuinely ordered (small, medium, large). Second mistake: leaving features in raw units; see the scaling experiment in Section 3.

One-hot versus integer coding

	One-hot	Integer code
columns	K	1
distances between categories	all equal ($\sqrt{2}$)	$1, 2, \ldots$: an invented order
when right	unordered categories	genuinely ordered levels
failure	none for distance methods; many columns	fake geometry, wrong neighbours

Key insight. Object → chosen features → vector: both arrows are human decisions, and "similar" means whatever those decisions make it mean.

How the professor asks it

Q. *A colleague represents a network flow by (duration in ms, protocol coded as TCP= 1, UDP= 2, ICMP= 3). Explain what a nearest-neighbour detector will do with the protocol column and how you would fix it.*
A. The detector will treat UDP as "between" TCP and ICMP and ICMP as twice as far from TCP as UDP is: distances that have no meaning. Mechanism: Euclidean distance subtracts codes, so the arbitrary numbering becomes geometry. Consequence: flows are matched partly on an accident of coding. Fix: one-hot encode the protocol (three binary columns, every pair at distance $\sqrt{2}$), then scale the duration column so that one protocol difference and one typical duration difference count comparably, with the scale chosen on validation data.

Q. *Give an example where adding features makes the learned model worse, and explain the mechanism.*
A. Append 500 columns of random noise to a good word-count representation. The information is unchanged, but a flexible learner can fit the training labels partly through the noise columns, so the fitted rule changes from sample to sample; the validation error rises until the training set is large enough for the learner to discover that those columns carry nothing. The representation decides how much data a given error level costs.

2 Deciding when the world is known: the Bayes rule

Conditional distribution, action, decision loss, conditional risk, Bayes rule

Write X for the input and Y for the output regarded as random; $\mathbb{P}(X, Y)$ is the joint distribution (how likely each pair is in the world). After seeing $X = x$, the uncertainty that *remains* is the conditional distribution $\mathbb{P}(Y \mid X = x)$: two houses with the same description can still sell at different prices because something was not recorded. An **action** $a \in \mathcal{A}$ is what we do (a predicted number, a declared class, "raise the alarm"). The **decision loss** $\ell_{\text{dec}}(y, a)$ is the damage of doing a when the truth is y. The **conditional risk** of a at x is its average damage under the remaining uncertainty, and the **Bayes decision rule** picks the action with the smallest conditional risk:

$$R(a \mid x) = \mathbb{E}[\ell_{\text{dec}}(Y, a) \mid X = x], \qquad f^*(x) \in \arg\min_{a \in \mathcal{A}} R(a \mid x) \quad \text{[on your sheet]}$$

"Bayes" here means "optimal under a known probability distribution"; nothing is assumed about

priors on parameters.

Purpose. Define the target that every learning method tries to approximate, and make explicit that the target depends on the loss.

Impact. Change the loss and the optimal action changes, with the same world and the same probabilities. Change the probabilities (better information) and the actions change too; the rule itself never changes.

Pointwise optimality is global optimality

Claim: for any decision rule f, $\mathbb{E}[\ell_{\text{dec}}(Y, f^*(X))] \le \mathbb{E}[\ell_{\text{dec}}(Y, f(X))]$; the left side is the **Bayes risk**, the floor nobody beats.

1. Tower property: $\mathbb{E}[\ell_{\text{dec}}(Y, f(X))] = \mathbb{E}\big[\mathbb{E}[\ell_{\text{dec}}(Y, f(X)) \mid X]\big]$. Averaging over all pairs equals "average over Y at each fixed x, then average over x".
2. At a fixed x, $f(x)$ is one particular action, so by definition of a minimiser $\mathbb{E}[\ell_{\text{dec}}(Y, f(x)) \mid X = x] \ge \min_a \mathbb{E}[\ell_{\text{dec}}(Y, a) \mid X = x] = \mathbb{E}[\ell_{\text{dec}}(Y, f^*(x)) \mid X = x]$.
3. An inequality that holds at every x survives averaging over x. Done.

Why it matters: we may think about one input at a time. Doing the best thing at every x is the best thing overall.

Regression: which number to report

Squared loss selects the conditional mean

Let $m(x) = \mathbb{E}[Y \mid X = x]$. Claim: $\mathbb{E}[(Y - a)^2 \mid X = x] = \text{Var}(Y \mid X = x) + (m(x) - a)^2$ [on your sheet], so the best a is $m(x)$.

1. Add and subtract the mean: $Y - a = (Y - m(x)) + (m(x) - a)$.
2. Square: $(Y - a)^2 = (Y - m(x))^2 + 2(Y - m(x))(m(x) - a) + (m(x) - a)^2$.
3. Take the conditional expectation. First term: $\text{Var}(Y \mid X = x)$ by definition. Middle term: $(m(x) - a)$ is a constant and $\mathbb{E}[Y - m(x) \mid X = x] = 0$, so it vanishes. Last term: a constant.
4. The variance does not depend on a; $(m(x) - a)^2 \ge 0$ with equality only at $a = m(x)$.

Reading: average squared error = unavoidable spread + your own offset squared. The spread is the problem's; the offset is yours. The move "add and subtract a mean, square, watch the cross term die" is reused in the bias–variance decomposition of Chapter 3.

Absolute loss selects a median; asymmetric loss selects a quantile

A **median** m satisfies $\mathbb{P}(Y \le m \mid x) \ge \frac{1}{2}$ and $\mathbb{P}(Y \ge m \mid x) \ge \frac{1}{2}$. Take $a > m$ and compare the two losses for each possible y (everything conditional on x):

- $y \le m$: $|y - a| - |y - m| = a - m$;
- $y \ge a$: $|y - a| - |y - m| = -(a - m)$;
- $m < y < a$: $|y - a| - |y - m| = a + m - 2y > -(a - m)$.

So $|y - a| - |y - m| \ge (a - m)[\mathbf{1}(y \le m) - \mathbf{1}(y > m)]$. Take expectations: $\mathbb{E}|Y - a| - \mathbb{E}|Y - m| \ge (a - m)[2\mathbb{P}(Y \le m) - 1] \ge 0$ because $\mathbb{P}(Y \le m) \ge \frac{1}{2}$. The case $a < m$ is symmetric. No a beats a median. [not on your sheet: know it](the sheet only states the result)

For the **asymmetric** loss $\ell_\tau(y, a) = \tau(y - a)$ if $y \ge a$ (under-prediction costs τ per unit) and $(1-\tau)(a-y)$ if $y < a$, the same three cases give $\mathbb{E}\ell_\tau(Y, a) - \mathbb{E}\ell_\tau(Y, m) \ge (a-m)[\mathbb{P}(Y \le m) - \tau] \ge 0$

whenever m is a τ-quantile ($\mathbb{P}(Y \le m) \ge \tau$, $\mathbb{P}(Y \ge m) \ge 1-\tau$). With costs c_u per unit under and c_o per unit over, $\tau = c_u/(c_u + c_o)$ [on your sheet]. $\tau = \frac{1}{2}$ is the median.

A delivery promise: three losses, three answers, one distribution

A courier must promise a delivery time for a route whose actual time is 100 minutes with probability 0.5, 200 with probability 0.3 and 600 with probability 0.2 (occasional road closures).

- Squared loss: promise the mean, $0.5(100) + 0.3(200) + 0.2(600) = 230$. Check: risk at 230 is $0.5(130)^2 + 0.3(30)^2 + 0.2(370)^2 = 36{,}100$; at 200 it is 37,000. The mean wins, although 230 never happens.
- Absolute loss: $\mathbb{P}(Y \le 100) = 0.5$ and $\mathbb{P}(Y \le 200) = 0.8$, $\mathbb{P}(Y \ge 200) = 0.5$, so every value in $[100, 200]$ is a median; the expected absolute loss is 130 for all of them ($a = 100$: $0.3(100) + 0.2(500)$; $a = 200$: $0.5(100) + 0.2(400)$) and 148 at the mean 230. The rare 600 drags the mean but barely moves the median.
- Being late costs 9 times being early: $\tau = 0.9$, so promise the 0.9-quantile, 600. The company prefers to disappoint nobody.

Nothing about the world changed; only the consequences did.

Classification: which class to declare

Zero–one loss selects the most probable class; the Bayes error

With $\ell_{\text{dec}}(y, a) = \mathbf{1}(y \ne a)$, the conditional risk of declaring a is $\mathbb{E}[\mathbf{1}(Y \ne a) \mid X = x] = \mathbb{P}(Y \ne a \mid X = x) = 1 - \mathbb{P}(Y = a \mid X = x)$. Minimising $1 - \mathbb{P}(Y = a \mid x)$ is maximising $\mathbb{P}(Y = a \mid x)$:

$$f^*(x) = \arg\max_c \mathbb{P}(Y = c \mid X = x) \quad \text{[on your sheet]}$$

$$\text{conditional risk } 1 - \max_c \mathbb{P}(Y = c \mid X = x)$$

Averaged over x this is the **Bayes error**, a property of the *representation*, not of any learner: with class probabilities $(0.5, 0.3, 0.2)$ at some x, the best rule errs half the time there and nothing can do better unless x is enriched. "Predict the most likely class" is the Bayes rule *only* for zero–one loss.

Cost-sensitive threshold for a binary decision

Class 1 is the event to detect, $p(x) = \mathbb{P}(Y = 1 \mid X = x)$, a false positive costs C_{FP}, a false negative C_{FN}, correct decisions cost 0. Declaring positive is wrong exactly when $Y = 0$, declaring negative exactly when $Y = 1$:

$$R(\text{positive} \mid x) = C_{\text{FP}}\,[1 - p(x)], \qquad R(\text{negative} \mid x) = C_{\text{FN}}\,p(x).$$

Positive is cheaper iff $C_{\text{FP}}(1-p) < C_{\text{FN}}\,p \iff C_{\text{FP}} < (C_{\text{FP}} + C_{\text{FN}})\,p$, i.e.

$$\text{declare positive iff } p(x) > t = \frac{C_{\text{FP}}}{C_{\text{FP}} + C_{\text{FN}}} \quad \text{[on your sheet]}$$

Equal costs give $\frac{1}{2}$. C_{FP} sits on top: expensive misses (large C_{FN}) give a *small* threshold and more alarms. Intrusion detection with $C_{\text{FN}} = 10$, $C_{\text{FP}} = 1$: $t = 1/11 \approx 0.09$; at $p = 0.2$ alarming costs 0.8, silence costs 2: alarm, although an attack is unlikely.

A third action, review, and why one threshold stops being enough

Add "send to an analyst", which resolves the case at a fixed cost C_{R} whatever the truth: $R(\text{review} \mid x) = C_{\mathrm{R}}$, a horizontal line in the plot of conditional risk against p. Each action is optimal where its line is lowest:

$$\begin{aligned} \text{negative iff } C_{\mathrm{FN}}\, p \le C_{\mathrm{R}} &\iff p \le C_{\mathrm{R}}/C_{\mathrm{FN}}, \\ \text{positive iff } C_{\mathrm{FP}}(1-p) \le C_{\mathrm{R}} &\iff p \ge 1 - C_{\mathrm{R}}/C_{\mathrm{FP}}, \\ \text{review iff } C_{\mathrm{R}}/C_{\mathrm{FN}} < p &< 1 - C_{\mathrm{R}}/C_{\mathrm{FP}} \quad \text{[on your sheet]} \end{aligned}$$

The two sloped lines cross once at $p^* = C_{\mathrm{FP}}/(C_{\mathrm{FP}} + C_{\mathrm{FN}})$, where their common height is $C_{\mathrm{FP}}C_{\mathrm{FN}}/(C_{\mathrm{FP}} + C_{\mathrm{FN}})$; the flat review line cuts the lower envelope twice iff C_{R} is below that peak, so a review band exists iff $C_{\mathrm{R}} < C_{\mathrm{FP}}C_{\mathrm{FN}}/(C_{\mathrm{FP}} + C_{\mathrm{FN}})$ [on your sheet]. With $C_{\mathrm{FP}} = 2$, $C_{\mathrm{FN}} = 8$, $C_{\mathrm{R}} = 0.9$ (Tutorial 1, Q2): benign for $p < 0.1125$, attack for $p > 0.55$, review between; the peak is 1.6 at $p^* = 0.2$. Cheaper review widens the band; raising C_{R} to 1.6 closes it.

The threshold-versus-cost experiment

Setup. A validation set of 10,000 flows with labels and, for each flow, a probability $p(x)$ from a calibrated model. Costs $C_{\mathrm{FP}} = 1$ and $C_{\mathrm{FN}} \in \{1, 3, 10, 30, 100\}$.

Held fixed. The model and its probabilities, the validation flows, C_{FP}.

What we manipulate. The threshold t from 0 to 1 in steps of 0.01, and separately the cost ratio.

What we measure. For each t: the number of false positives and false negatives on the validation set and the total cost $C_{\mathrm{FP}} \cdot \mathrm{FP}(t) + C_{\mathrm{FN}} \cdot \mathrm{FN}(t)$.

What the plot shows. Total cost against t, one curve per cost ratio. Each curve is U-shaped; its minimum sits at $t \approx C_{\mathrm{FP}}/(C_{\mathrm{FP}} + C_{\mathrm{FN}})$: $0.5, 0.25, 0.09, 0.03, 0.01$. As C_{FN} grows the minimum slides left and the curve becomes steep on the right (missing attacks is what hurts).

How to read it. The probabilities were learned once; the *policy* is a dial on top of them. Changing costs moves the operating point, never the model. If the minimum is not near the formula's value, the probabilities are not calibrated (Chapter 4).

Triage: the most probable class loses

A patient is healthy, mildly ill or severely ill with probabilities $(0.5, 0.3, 0.2)$. Actions: discharge, monitor, treat. Costs (rows truth, columns action): healthy $(0, 1, 3)$, mild $(4, 0, 2)$, severe $(20, 6, 0)$. Conditional risks are dot products of the probability row with each column: discharge $0.5(0) + 0.3(4) + 0.2(20) = 5.2$; monitor $0.5(1) + 0 + 0.2(6) = 1.7$; treat $0.5(3) + 0.3(2) + 0 = 2.1$. Bayes decision: **monitor**, although "healthy" is the most probable state. The 20 in the corner (discharging a severe case) rules the calculation. Other real cases: card fraud with $C_{\mathrm{FP}} = 5$ (annoyed customer) and $C_{\mathrm{FN}} =$ the amount gives a threshold $5/(5 + A)$ that depends on x; a court sets the threshold far above $\frac{1}{2}$ ("beyond reasonable doubt"); medical screening sets it below.

Inverting the threshold; predicting the mode under every loss

The threshold is $C_{\mathrm{FP}}/(C_{\mathrm{FP}} + C_{\mathrm{FN}})$, not $C_{\mathrm{FN}}/(\cdot)$ and not $C_{\mathrm{FP}}/C_{\mathrm{FN}}$ (that is an odds). Check with the words: expensive misses must give *more* alarms, so a smaller threshold. Second mistake: "predict the most likely value" regardless of loss. Under squared loss the answer is the mean, which may be a value that never occurs; under absolute loss the median; under zero–one the mode. Third: the Bayes rule needs the *true* $\mathbb{P}(Y \mid X = x)$; with an estimate in its place, the decision inherits every error of the estimate, which is why calibration (Chapter 4) matters for thresholds and review bands.

The four losses and what each one selects

Loss	Optimal action	Sensitive to	Typical use
squared $(y-a)^2$	conditional mean	outliers (they pull the mean)	symmetric numeric errors
absolute $\lvert y-a\rvert$	conditional median	little; robust	robust point prediction
asymmetric, c_u under / c_o over	τ-quantile, $\tau = c_u/(c_u+c_o)$	the cost ratio	stock, delivery promises
zero–one $\mathbf{1}(y \neq a)$	most probable class	nothing but the ranking	equal-cost classification
cost table $C_{\mathrm{FP}}, C_{\mathrm{FN}}$	declare positive iff $p > C_{\mathrm{FP}}/(C_{\mathrm{FP}}+C_{\mathrm{FN}})$	the cost ratio	detection, screening

Key insight. Probabilities describe uncertainty, the loss describes consequences, and only together do they determine an action; "best prediction" has no meaning until the loss is specified.

How the professor asks it

Q. *A spam filter estimates $p(x) = 0.3$ for an email. Deleting a legitimate email costs $C_{\mathrm{FP}} = 7$, letting spam through costs $C_{\mathrm{FN}} = 3$. Derive the conditional risks of both actions and decide; then explain in one sentence why the decision would flip if the costs were swapped.*
A. Keep the email. Deleting is wrong with probability 0.7 and costs $7 \times 0.7 = 4.9$; keeping is wrong with probability 0.3 and costs $3 \times 0.3 = 0.9$. The threshold is $7/10 = 0.7$ and $0.3 < 0.7$. With costs swapped the threshold is $3/10 = 0.3$ and the email sits exactly at it: either action is optimal, and any p above 0.3 would be deleted. The probability did not change; the policy did.

Q. *Explain why the Bayes error is a property of the representation and not of the learning algorithm, and what a team should do when the Bayes error is too high for the application.*
A. At a fixed x the best possible rule declares the majority class and still loses the minority share $1 - \max_c \mathbb{P}(Y = c \mid x)$; averaged over x that is the Bayes error. No learner can beat it because it is computed from the true conditional probabilities, which no learner knows better than the world. Its size is set by how much of Y the recorded features explain. If it is too high, the fix is a richer x (more or better features, a different sensor), not a better algorithm or relabelling.

Q. *A delivery time is 20 minutes with probability 0.6 and 80 minutes with probability 0.4. What should be promised under squared loss, under absolute loss, and when a late delivery costs four times an early one? Explain each choice.*
A. Squared loss: the mean $0.6(20) + 0.4(80) = 44$, because squared error splits into variance plus squared offset from the mean. Absolute loss: a median; $\mathbb{P}(Y \leq 20) = 0.6 \geq \frac{1}{2}$ and $\mathbb{P}(Y \geq 20) = 1$, so 20 (any value in $[20, 20]$ only). Late costs 4, early costs 1: $\tau = 4/5 = 0.8$, and we need $\mathbb{P}(Y \leq m) \geq 0.8$, which first holds at $m = 80$: promise 80. Three promises from one distribution; the loss chose.

3 From the known distribution to finite data: nearest neighbours

Training dataset, the i.i.d. assumption, the plug-in idea

We never know $\mathbb{P}(X, Y)$. What we have is a **training dataset** $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$ [on your sheet]: N rows, superscript (i) indexes examples, subscript j indexes features, so $x_j^{(i)}$ is feature j of example i. The standard assumption is that the rows are **independent and identically distributed** draws from $\mathbb{P}(X, Y)$: like N tickets drawn independently from one urn. It fails for consecutive days of a time series (dependent), for houses in one street (dependent), and when the market

changes (a different urn: distribution shift). The task of learning is to build, from the sample, a rule that approximates what we would want under the unknown distribution. The most direct way is the **plug-in idea**: estimate $\mathbb{P}(Y \mid X = x)$ or $\mathbb{E}[Y \mid X = x]$ from the sample and feed the estimate to the Bayes rule of Section 2 unchanged.

Purpose. Separate the two objects that students confuse: the true distribution (unknown, what we want) and the sample (observed, what we have).

Impact. Every guarantee of this chapter (honest estimates, convergence) rests on i.i.d.; grouped or temporal data break it and need a different split (Chapter 3).

k-nearest neighbours

If many examples had exactly the input x we would average their outputs; exact repeats are rare, so use the k examples whose representations are *closest*. With $N_k(x)$ the indices of the k nearest training inputs under a chosen distance,

$$\hat{f}_k(x) = \frac{1}{k} \sum_{i \in N_k(x)} y^{(i)}, \qquad \hat{p}_k(Y = c \mid x) = \frac{1}{k} \sum_{i \in N_k(x)} \mathbf{1}(y^{(i)} = c) \quad \text{[on your sheet]}$$

Regression averages the neighbours' values; classification reports the *fraction* of neighbours in each class, and under zero–one loss declares the majority (under costs, feed $\hat{p}_k$ to the threshold). kNN is **lazy**: nothing is fitted at training time; the data are the model. It has *no parameters* and three **hyperparameters**: k, the distance, and the feature scales. For $k = 1$ the input space is cut into **Voronoi cells**, one per training point, and the boundary between two points is their perpendicular bisector.

Purpose. Imitate the Bayes rule locally with no assumption on the shape of $\mathbb{P}(Y \mid X)$.

Impact. Small k follows individual points (low training error, high sensitivity to noise); large k averages over a wide region (smoother, can erase genuine local variation). Capacity *rises* as k *falls*; $k = N$ gives the global average everywhere.

kNN by hand: the decision flips with k

Six training points with class and value: 1:$(1, 1)$ A 10; 2:$(2, 1)$ A 12; 3:$(1, 3)$ B 20; 4:$(3, 3)$ B 22; 5:$(4, 1)$ A 14; 6:$(3, 2)$ B 18. Query $x = (2, 1.8)$. Squared Euclidean distances (same ordering as Euclidean, no roots): to 1: $1 + 0.64 = 1.64$; 2: 0.64; 3: 2.44; 4: 2.44; 5: 4.64; 6: $1 + 0.04 = 1.04$. Order of closeness $2, 6, 1, \{3, 4\}, 5$. $k = 1$: neighbour $\{2\}$, class A, value 12. $k = 3$: $\{2, 6, 1\}$, classes A, B, A, $\hat{p}(\text{A}) = 2/3$, class A, value $(12 + 18 + 10)/3 = 13.3$. $k = 5$: $\{2, 6, 1, 3, 4\}$, A, B, A, B, B, $\hat{p}(\text{B}) = 3/5$, class **B**, value 16.4. $k = 6$: a 3:3 tie, value 16, the global mean. The answer changed from A to B as k grew: that is what k manipulates.

Vary k and plot training and validation error

Setup. One labelled dataset, standardised features, a fixed random split into training and validation (or K folds).

Held fixed. The data, the split, the distance.

What we manipulate. $k \in \{1, 3, 5, 9, 15, 25, 51, 101, N\}$.

What we measure. For each k, the error of the kNN rule on the training points themselves and on the validation points.

What the plot shows. Two curves against k, with k increasing to the right. Training error starts at exactly 0 for $k = 1$ (every point is its own nearest neighbour) and rises with k. Validation error is high at $k = 1$ (each prediction trusts one noisy neighbour), falls to a minimum at moderate k, then rises again towards the error of "predict the global majority" at $k = N$. Read with the

capacity axis reversed: small k is high capacity.

How to read it. The k with the lowest *validation* error is the one to keep; training error would always pick $k = 1$. The gap between the curves at small k is overfitting; both curves high at large k is underfitting. With more data the minimum moves to larger k (more neighbours are genuinely close).

Distances: Minkowski, Euclidean, Manhattan, Hamming, scaled and weighted

For $x, z \in \mathbb{R}^d$ and $p \geq 1$, $d_p(x, z) = \left(\sum_j |x_j - z_j|^p\right)^{1/p}$ [on your sheet]: $p = 1$ Manhattan (sum of coordinate gaps, "along the streets"), $p = 2$ Euclidean (straight line), $p \to \infty$ the largest gap. For binary vectors the **Hamming** distance counts differing positions (10110 versus 11100: 2). The **scaled, weighted** Euclidean distance $d_w(x, z) = \sqrt{\sum_j w_j ((x_j - z_j)/s_j)^2}$ [on your sheet]divides each gap by a scale s_j (units, typical spread) and multiplies by a weight $w_j \geq 0$ ($w_j = 0$ ignores feature j). For $x = (1, 2)$, $z = (3, 5)$: gaps $2, 3$; $d_1 = 5$, $d_2 = \sqrt{13} \approx 3.61$, $d_\infty = 3$.

Purpose. Define "nearest". The distance *is* the inductive bias of kNN.

Impact. Scales and weights change who the neighbours are, hence the prediction, without touching the data; they are hyperparameters chosen on validation.

The scaling experiment: same customers, different neighbour

Setup. Two training customers described by (income in USD, age in years): $P = (30{,}000, 25)$ and $Q = (31{,}000, 60)$. Query: a 25-year-old earning 30,800.

Held fixed. The three people.

What we manipulate. The units: raw, then standardised with typical spreads $s_{\text{income}} = 1000$, $s_{\text{age}} = 10$.

What we measure. The nearest neighbour.

What the plot shows. Raw: distance to P is $\sqrt{800^2 + 0} = 800$, to Q is $\sqrt{200^2 + 35^2} \approx 203$: nearest is Q, a 60-year-old, because a few hundred dollars dwarf 35 years *in these units*. Standardised: $P \to (30.0, 2.5)$, $Q \to (31.0, 6.0)$, query $\to (30.8, 2.5)$; distance to P is 0.8, to Q is $\sqrt{0.04 + 12.25} \approx 3.5$: nearest is P, the customer of the same age.

How to read it. Nothing about the customers changed; the choice of units silently chose the answer. Standardise each feature with $z_j = (x_j - \mu_j)/\sigma_j$, with μ_j, σ_j computed on the *training* rows only and reused for every new input.

The curse of dimensionality, two ways

Grid count. Covering $[0, 1]^d$ with 10 positions per coordinate needs 10^d cells (each coordinate chosen independently among 10): 10^2 cells for $d = 2$, 10^{10} for $d = 10$. A sample that fills the grid in two dimensions is empty in ten.

Probability of a true neighbour. Put n uniform points in the unit ball of $\mathbb{R}^d$, query at the centre, "true neighbour" means within radius $r = 0.1$. A ball of radius r has volume r^d times the unit ball's (scaling each coordinate by r scales d-dimensional volume by r^d), so one point is a neighbour with probability 0.1^d; the count is $M_d \sim \text{Binomial}(n, 0.1^d)$, and independence gives

$$\mathbb{P}(M_d \geq 1) = 1 - (1 - 10^{-d})^n \approx 1 - e^{-n\,10^{-d}} \quad \text{[on your sheet]}$$

With $n = 1000$:

d	1	2	3	4	5	6	10
$\mathbb{P}(\geq 1$ neighbour)	≈ 1	0.99996	0.632	0.095	0.010	0.001	10^{-7}

$d = 3$: $1 - e^{-1}$; $d = 4$: $1 - e^{-0.1}$; expected count $n\,10^{-d}$ is 1 at $d = 3$ and 0.1 at $d = 4$. The sheet's other form, the radius needed for a 95% chance of one neighbour, $r_d = (1-0.05^{1/n})^{1/d} \to 1$ [on your sheet], says the same thing: in high dimension the "neighbourhood" is the whole ball. The calculation flatters kNN: the query sits at the centre (most volume is near the surface, where half the neighbourhood is empty), one neighbour is counted as enough (averaging label noise needs k), and every dimension is assumed relevant.

The curse experiment: dimension against the chance of a neighbour

Setup. Draw $n = 1000$ points uniformly in the unit ball of $\mathbb{R}^d$; query at the centre.

Held fixed. n, the radius $r = 0.1$ that defines "genuinely close".

What we manipulate. $d = 1, 2, \ldots, 10$; repeat each d 1000 times.

What we measure. The fraction of repetitions in which at least one point lies within r of the query; also the distance to the nearest point.

What the plot shows. The fraction against d: flat at 1 for $d \leq 2$, 0.63 at $d = 3$, 0.10 at $d = 4$, essentially 0 from $d = 6$. The nearest-point distance rises with d towards the radius of the whole ball.

How to read it. The "neighbours" kNN averages in high dimension are far away, so the local average is no longer an estimate of anything local. Local estimation cannot exploit many features without extra structure; that is the motivation for a shared parametric model (Section 4). Remedies that stay with kNN: select features, learn a representation, or reweight dimensions.

Intrusion detection with 80 flow statistics

A team describes each flow by 80 statistics and runs 5-NN on 50,000 flows. In 80 dimensions no stored flow is genuinely close to a new one, so the five "neighbours" merely share the dominant large-scale features (duration, bytes) and the detector degrades to a crude rule on those. Standardising helps, selecting the 8 features that matter helps more, and a model that shares parameters across all flows (Chapters 4 and 5) is what gets deployed. Cost also bites: 5-NN stores all N rows ($O(Nd)$ memory) and computes N distances per query ($O(Nd)$ per prediction), slow at line rate.

Choosing k or the scales on the training error, or on the test set

Tempting because the training error is available immediately. But the 1-NN training error is 0 by construction (each point is its own neighbour), so training error always picks $k = 1$ and says nothing about new flows. Tuning the scale w on the query you are being asked about, or on the test set, fits the metric to the evaluation. Produces: an optimistic error and a k that overfits. Fix: choose k, the distance and the scales on held-out validation data (or by cross-validation), then evaluate once on the test set. Second mistake: computing μ_j, σ_j on all rows before splitting; that leaks test statistics into the representation.

kNN versus a parametric model

	kNN	Parametric model f_θ
what is learned	nothing; the data are the model	θ, from all examples jointly
assumption	nearby inputs have similar outputs	the family $\mathcal{H}$ contains a good rule
what we manipulate	k, distance, scales	family, hyperparameters, θ
training cost	zero	an optimisation
prediction cost	$O(Nd)$ per query	usually $O(d)$
fails when	d large relative to N (curse), unscaled features	the family is wrong (bias)
prefer when	low d, plenty of data, irregular structure	high d, structure that generalises across inputs

Key insight. kNN is the Bayes rule estimated locally; "local" is defined by the distance you chose and stops existing when the dimension is large.

How the professor asks it

Q. *Tutorial 1, Q3: a query flow $x = (50\ ms, 10\ packets)$ has neighbours $x_A = (60, 100)$ labelled attack and $x_B = (400, 12)$ labelled benign. Which is nearest in raw units, which after scaling duration by 100 and packets by 20, and what does the disagreement teach?*
A. Raw: $d(x, x_A) = \sqrt{10^2 + 90^2} \approx 90.6$, $d(x, x_B) = \sqrt{350^2 + 2^2} \approx 350.0$: A is nearest, so attack. Scaled: $z = (0.5, 0.5)$, $z_A = (0.6, 5.0)$, $z_B = (4.0, 0.6)$; $d = \sqrt{0.01 + 20.25} \approx 4.50$ to A and $\sqrt{12.25 + 0.01} \approx 3.50$ to B: B is nearest, so benign. The flows did not change; the metric did, because milliseconds produce the large numbers in raw units. Consequence: scales are hyperparameters, chosen on validation flows, never on the query or the test set.

Q. *Exercises 2, Q6: with $n = 1000$ uniform points in the unit ball and "neighbour" meaning within radius 0.1, find the first dimension at which the probability of at least one neighbour drops below one half, and name three ways the calculation still flatters kNN.*
A. $d = 4$: $\mathbb{P} = 1 - (1 - 10^{-4})^{1000} \approx 1 - e^{-0.1} = 0.095$, against 0.632 at $d = 3$. Mechanism: the volume of the 0.1-ball is 10^{-d} of the unit ball, so each point is a neighbour with that probability. It flatters because the query sits at the centre (near the surface half the neighbourhood is outside the ball), one neighbour is counted as enough although averaging noise needs k, and all d dimensions are assumed relevant whereas irrelevant ones spend the radius on noise.

Q. *Why is the training error of 1-NN zero, and why does this not make 1-NN a good classifier?*
A. Each training point is its own nearest neighbour, so it receives its own label: zero error by construction. This measures memorisation, not the population risk; a 1-NN rule copies every mislabelled or noisy point into its Voronoi cell. If k were chosen by training error it would always be 1, which is exactly why k must be chosen on data the rule did not see.

4 Parameterised models: families of rules

Parameterised model, hypothesis space, nested families

A **parameterised model** is a function $f_\theta(x)$ whose behaviour is set by adjustable numbers $\theta \in \Theta$, the **parameters**. The set of all functions it can produce,

$$\mathcal{H} = \{f_\theta : \theta \in \Theta\} \quad \text{[on your sheet]}$$

is the **hypothesis space**. The model *form* fixes how parameters are shared across inputs; θ picks one member. Linear regression: $f_\theta(x) = w^\top x + b$, $\theta = (w, b)$; with $w = (2, -1)$, $b = 3$

and $x = (1, 4)$, $f_\theta(x) = 2 - 4 + 3 = 1$. Every member is a flat surface: no setting of the dials produces a curve, and that restriction is the **inductive bias** of linear regression. Polynomials of degree $\leq p$ in one variable: changing coefficients selects another polynomial of the same degree; changing p changes the *family*. Families are **nested** when $\mathcal{H}_1 \subseteq \mathcal{H}_2$ (degree $\leq p$ inside degree $\leq p+1$: set $\theta_{p+1} = 0$).

Purpose. Share structure across inputs so that all N examples inform the prediction at every x, which is what kNN could not do in high dimension.

Impact. Choosing $\mathcal{H}$ decides what can be expressed before any data are seen; a wrong family is an error that more data never removes.

Probability simplex, probabilistic classifier, softmax

A classifier for C classes estimates the vector of conditional probabilities $\hat{p}_\theta(x) = [\hat{p}_\theta(Y = 1 \mid x), \ldots, \hat{p}_\theta(Y = C \mid x)]^\top$ in the **probability simplex** $\Delta^{C-1} = \{p \in \mathbb{R}^C : p_c \geq 0,\ \sum_c p_c = 1\}$: all legitimate probability assignments. $(0.5, 0.3, 0.2)$ is in it; $(0.5, 0.5, 0.5)$ and $(1.2, -0.2, 0)$ are not. The usual construction computes C real **scores** $s_{\theta,c}(x)$ and maps them by the **softmax** $p_c = e^{s_c} / \sum_{c'} e^{s_{c'}}$ [on your sheet]: scores $(2, 0, -1)$ become $(7.39, 1, 0.37)/8.76 \approx (0.84, 0.11, 0.04)$. A decision rule of Section 2 then turns the vector into an action.

Purpose. Keep prediction (a probability vector) separate from decision (an action), so one model serves every cost structure.

Impact. The model outputs are what thresholds, review bands and prior corrections (Chapter 4) act on; if they are not calibrated, every policy built on them is off.

Parameters versus hyperparameters

Parameters are learned from the training examples by the learning algorithm. **Hyperparameters** are choices made around that estimation: some define the family (polynomial degree, number of layers), some control the procedure (number of neighbours k, step sizes, feature scales):

$$\text{model form} + \text{hyperparameters} \to \mathcal{H} \to \theta \to \text{one particular model.}$$

Model	Parameters (learned)	Hyperparameters (chosen)
kNN	none	k, distance, feature scales
linear regression	w, b	which features to include
polynomial regression	coefficients $\theta_0, \ldots, \theta_p$	degree p
neural network	all weights	layers, width, learning rate

Purpose. Separate what the data decide (inside a fixed configuration) from what we decide (which configuration), because the two need different evidence (Section 6).

Impact. The same quantity can be either: the degree is a hyperparameter if we fix it and fit coefficients, a parameter if a procedure searches over it with its own criterion. What makes it one or the other is the *procedure*, not the symbol.

Letting the training data choose a hyperparameter

kNN with k picked by training error returns $k = 1$ every time; a polynomial degree picked by training error returns the largest degree offered (Section 5). The mechanism is the same: the training score can only reward flexibility. Produces: a model that memorises. Fix: hyperparameters are chosen on validation data, parameters on training data.

Key insight. A hypothesis space is a bet placed before seeing data; parameters are what the data adjust inside that bet, hyperparameters are the bet itself.

How the professor asks it

Q. *Exercises 2, Q8: is the degree of a polynomial regression a parameter or a hyperparameter? Answer with the criterion that decides, and give a procedure under which the answer changes.*
A. It depends on the procedure. If we fix the degree and fit the coefficients by least squares on the training set, the degree is a hyperparameter: chosen by us, outside the fitting criterion, on validation data. If instead a procedure searches jointly over degree and coefficients with a single criterion that penalises degree (a complexity-penalised risk), the degree is being estimated from data by that criterion and is a parameter of that procedure. The dividing line is who chooses it and on which data, not what it is called.

Q. *Explain why a classifier that outputs scores needs a map to the simplex before any decision, and what goes wrong if scores are compared with a cost threshold directly.*
A. A cost threshold $C_{\mathrm{FP}}/(C_{\mathrm{FP}}+C_{\mathrm{FN}})$ is a statement about probabilities: it compares $C_{\mathrm{FP}}(1-p)$ with $C_{\mathrm{FN}}p$. Raw scores have no scale (adding a constant or doubling them changes nothing about the ranking), so comparing them with 0.2 is meaningless. Softmax (or the sigmoid) turns scores into nonnegative numbers summing to one; only then do the conditional risks of Section 2 make sense. In the binary case the equivalent operation on the score is to compare s with $\log \frac{t}{1-t}$ (Chapter 4).

5 Learning the parameters from data: ERM and cross-entropy

Two losses, two jobs; population risk, empirical risk, ERM

Two losses do two jobs: ℓ_{train} guides the *fitting* of the model to data, ℓ_{dec} guides the *action* taken from the fitted model's output. They may coincide (squared loss to fit and to judge a regression) or differ (cross-entropy to fit probabilities, zero–one or costs to decide). For parameters θ:

$$R(\theta) = \mathbb{E}_{(X,Y)\sim\mathbb{P}}\big[\ell_{\mathrm{train}}(Y, f_\theta(X))\big] \text{ (population risk)},$$

$$\widehat{R}_{\mathrm{train}}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \ell_{\mathrm{train}}(y^{(i)}, f_\theta(x^{(i)})) \text{ (empirical risk)} \quad \text{[on your sheet]}$$

R is the average loss over every input the world will ever produce: what we want, unknowable. $\widehat{R}$ is the average over our N rows: computable (three examples with squared errors $1, 4, 0$ give $5/3$). **Empirical risk minimisation** picks $\hat{\theta} \in \arg\min_\theta \widehat{R}_{\mathrm{train}}(\theta)$ [on your sheet]; the hat says "computed from this sample", and another sample gives another $\hat{\theta}$. The best population parameters $\theta^*_{\mathcal{H}} \in \arg\min_\theta R(\theta)$ may still have risk above the Bayes risk if $\mathcal{H}$ does not contain the Bayes rule.

Purpose. Replace an average over the unknown world by an average over the sample, the only thing we can compute.

Impact. Everything that can go wrong in learning lives in the gap between R and $\widehat{R}$: the next two results say when the gap is harmless and when it is not.

Honest for a fixed model, optimistic after minimisation

Fixed model. Fix θ without looking at the data. Each term $\ell_{\mathrm{train}}(y^{(i)}, f_\theta(x^{(i)}))$ is an independent copy of the random variable $\ell_{\mathrm{train}}(Y, f_\theta(X))$ whose mean is $R(\theta)$; the expectation of an average of such terms is $R(\theta)$. So $\mathbb{E}[\widehat{R}_{\mathrm{train}}(\theta)] = R(\theta)$ and, by the law of large numbers, $\widehat{R}_{\mathrm{train}}(\theta) \to R(\theta)$ as $N \to \infty$. [not on your sheet: know it]
Fitted model. Let θ^* minimise R and $\hat{\theta}$ minimise $\widehat{R}_{\mathrm{train}}$. Then

$$\mathbb{E}[\widehat{R}_{\mathrm{train}}(\hat{\theta})] \le \min_\theta R(\theta) \le \mathbb{E}[R(\hat{\theta})] \quad \text{[on your sheet]}$$

1. Because $\hat{\theta}$ minimises $\widehat{R}_{\text{train}}$, for every sample $\widehat{R}_{\text{train}}(\hat{\theta}) \le \widehat{R}_{\text{train}}(\theta^*)$.
2. Take expectations; θ^* is fixed, so the right side has expectation $R(\theta^*) = \min_\theta R$ by the first part.
3. $R(\hat{\theta}) \ge \min_\theta R$ holds for every sample, hence in expectation.

Reading: the fitted model was chosen *because* it looks good on these rows; part of its score is real, part is luck that will not recur. The training score is at least as flattering as the truth, on average.

The honest-versus-optimistic experiment

Setup. A simulated world where you can draw as many samples as you like (say $Y = 2x + \varepsilon$). Pick a family (lines) and one fixed line θ_0 chosen *before* seeing any sample.

Held fixed. The world, the family, θ_0, the sample size $N = 20$.

What we manipulate. Repeat 1000 times: draw a fresh training sample; compute (a) the training risk of the fixed line θ_0, (b) the training risk of the least-squares line $\hat{\theta}$ fitted on that sample, (c) the true risk $R(\hat{\theta})$ of that fitted line (computable because the world is simulated, or from a huge fresh sample).

What we measure. The averages of (a), (b), (c) over the 1000 repetitions, and the histograms.

What the plot shows. Three histograms. (a) is centred on $R(\theta_0)$: honest. (b) is centred *below* $\min_\theta R$: optimistic. (c) is centred *above* $\min_\theta R$: the fitted line is a little worse than the best line, because it chased the sample's noise. The ordering (b) < best < (c) is the proposition, seen as data.

How to read it. The same number, "average loss on the training rows", is an honest estimate for a rule chosen in advance and a flattering one for the rule the rows chose. Shrinking N widens the gap; growing N closes it.

Bigger families fit at least as well; enough flexibility fits anything

Nested families. If $\mathcal{H}_1 \subseteq \mathcal{H}_2$ then $\min_{f \in \mathcal{H}_2} \widehat{R}_{\text{train}}(f) \le \min_{f \in \mathcal{H}_1} \widehat{R}_{\text{train}}(f)$ [on your sheet]: the minimum over a larger set is at most the minimum over a subset, because every candidate of $\mathcal{H}_1$ is still available in $\mathcal{H}_2$. Raising the polynomial degree never raises the best training risk.

Interpolation. Given N points with distinct inputs $x^{(1)}, \dots, x^{(N)} \in \mathbb{R}$, the Lagrange polynomial $L_i(x) = \prod_{j \ne i} \frac{x - x^{(j)}}{x^{(i)} - x^{(j)}}$ has degree $N - 1$, equals 1 at $x^{(i)}$ and 0 at every other $x^{(j)}$; so $f(x) = \sum_i y^{(i)} L_i(x)$ passes through all N points and has training risk 0 under any loss with $\ell(y, y) = 0$. [not on your sheet: know it]

Together: making the family more flexible can only lower the training risk, all the way to zero, even after performance on new inputs has started to deteriorate. Training risk is therefore a poor guide to *complexity*; parameters are estimated on training data inside a fixed configuration, configurations are compared on validation data.

Least squares by hand, and the nested families in action

Fit $f(x) = wx + b$ to $(0,1), (1,3), (2,2), (3,4)$ under squared loss. Setting $\partial \widehat{R}/\partial b = 0$ and $\partial \widehat{R}/\partial w = 0$ gives the normal equations $\sum y = w \sum x + Nb$ and $\sum xy = w \sum x^2 + b \sum x$, hence

$$w = \frac{N \sum xy - \sum x \sum y}{N \sum x^2 - (\sum x)^2}, \qquad b = \frac{\sum y - w \sum x}{N}. \quad \text{[not on your sheet: know it]}$$

Here $N = 4$, $\sum x = 6$, $\sum y = 10$, $\sum x^2 = 14$, $\sum xy = 19$: $w = (76 - 60)/(56 - 36) = 0.8$, $b = (10 - 4.8)/4 = 1.3$. Predictions $1.3, 2.1, 2.9, 3.7$; residuals $-0.3, 0.9, -0.9, 0.3$ (they sum to

zero, as $\partial\widehat{R}/\partial b = 0$ demands); training risk $(0.09 + 0.81 + 0.81 + 0.09)/4 = 0.45$. Degree 0: the mean 2.5, risk 1.25. Degree 3: a cubic through all four points, risk 0. The sequence $1.25, 0.45, 0$ is non-increasing, exactly as the nested-family result says, and it says nothing about which degree predicts a fifth point; Chapter 3 adds the point $(4, 5)$ and the cubic predicts 15.

Learning probabilities: why cross-entropy and not zero–one

Why the zero–one training risk cannot be optimised

The natural decision loss for classification is zero–one with decision $\arg\max_c \hat{p}_{\theta,c}(x)$. As a *training* loss, $\widehat{R}_{01}(\theta) = \frac{1}{N}\sum_i \mathbf{1}(y^{(i)} \neq \arg\max_c s_{\theta,c}(x^{(i)}))$ [on your sheet]is unusable: it ignores changes in confidence, it is piecewise constant in θ, and its gradient is 0 wherever it exists and undefined where a decision flips. One feature, threshold classifier $f_\theta(x) = \mathbf{1}(x > \theta)$, data $x = 1, 2, 3, 4$ with labels $0, 0, 1, 1$: $\widehat{R}_{01}$ is $\frac{2}{4}$ for $\theta < 1$, $\frac{1}{4}$ on $[1, 2)$, 0 on $[2, 3)$, $\frac{1}{4}$ on $[3, 4)$, $\frac{2}{4}$ beyond: a staircase. Between steps the slope is 0, so "move θ to decrease the loss" gets no information.

Purpose. Explain what a training loss must have: it must vary smoothly with the model's output and reward better probabilities, not just a correct argmax.

Cross-entropy training loss

With $\hat{p}_{\theta,c}(x)$ the model's probability for class c, the **cross-entropy** of an example is minus the log of the probability given to the class that actually occurred, and the empirical cross-entropy is its average:

$$\ell_{\mathrm{CE}}(y^{(i)}, \hat{p}_\theta(x^{(i)})) = -\log \hat{p}_{\theta,y^{(i)}}(x^{(i)}), \qquad \widehat{R}_{\mathrm{CE}}(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log \hat{p}_{\theta,y^{(i)}}(x^{(i)}) \quad \text{[on your sheet]}$$

Binary form with $\hat{p} = \hat{p}_\theta(Y = 1 \mid x)$ and $y \in \{0, 1\}$: $-y \log \hat{p} - (1 - y)\log(1 - \hat{p})$ [on your sheet]; only the term of the class that occurred survives. Natural logs: probability 0.9 on the true class costs 0.105; 0.6 costs 0.511; 0.5 costs 0.693; 0.1 costs 2.303; 0.01 costs 4.605. Three examples at $0.9, 0.6, 0.1$ give $\widehat{R}_{\mathrm{CE}} = (0.105 + 0.511 + 2.303)/3 \approx 0.973$: the one confident mistake dominates. Where it comes from: the one-hot vector of the observed label has cross-entropy $-\sum_c t_c \log \hat{p}_c = -\log \hat{p}_y$ with the model's vector.

Purpose. A smooth, graded training loss whose minimiser is the true conditional probability (next box), so the output can feed any decision rule afterwards.

Impact. Confident mistakes are punished hardest; hedging costs a little everywhere. Training with cross-entropy and deciding with costs is the standard division of labour.

Cross-entropy is minimised by the true probabilities, and equals maximum likelihood

Let $p_c(x) = \mathbb{P}(Y = c \mid X = x)$ be the truth. At a fixed x, $Y = c$ with probability $p_c(x)$, so the expected loss of the model's vector is

$$\mathbb{E}[-\log \hat{p}_{\theta,Y}(x) \mid X = x] = -\sum_c p_c(x) \log \hat{p}_{\theta,c}(x)$$
$$= H(p(x)) + \mathrm{KL}\left(p(x) \,\|\, \hat{p}_\theta(x)\right) \;\geq\; H(p(x)) \quad \text{[on your sheet]}$$

Steps: the first equality is the definition of expectation; the second is the identity cross-entropy = entropy + KL (add and subtract $\sum_c p_c \log p_c$); the inequality is Gibbs (KL ≥ 0, equality iff $\hat{p}_\theta(x) = p(x)$), itself from $\ln u \leq u - 1$. Averaging over X: $R_{\mathrm{CE}}(\theta) = H(Y \mid X) + \mathbb{E}_X[\mathrm{KL}(p(X) \| \hat{p}_\theta(X))]$. The first term is the problem's own noise, which no model removes; the second is *your* error, and training pushes only that down. Over all probability models the

minimiser outputs the true conditional probabilities: **calibration**. Inside a restricted family the minimiser is the member closest to the truth in averaged KL, which need not equal it.
Maximum likelihood. The conditional likelihood of the observed labels is $L(\theta) = \prod_i \hat{p}_{\theta,y^{(i)}}(x^{(i)})$ and $\log L(\theta) = \sum_i \log \hat{p}_{\theta,y^{(i)}}(x^{(i)}) = -N\widehat{R}_{\mathrm{CE}}(\theta)$ [on your sheet]; log is increasing and $-N < 0$, so maximising L and minimising $\widehat{R}_{\mathrm{CE}}$ have the same solutions.

Learning objective versus learning algorithm

ERM specifies an **objective**: which θ is preferred. A **learning algorithm** specifies how it is found: a closed form (least squares) or an iterative optimisation producing $\theta_0, \theta_1, \ldots$ (gradient methods, later in the course). The full chain separates learning, inference and decision:

$$\mathcal{D} \xrightarrow{\ell_{\text{train}}} f_{\hat{\theta}} \xrightarrow{\text{new } x} \text{inferred quantity } \hat{p}_{\hat{\theta}}(x) \xrightarrow{\ell_{\text{dec}}} \text{action.}$$

Purpose. Keep three questions apart: what is a good model (objective), how to find it (algorithm), what to do with its output (decision).

Impact. A bad algorithm can fail to reach the objective's minimiser (optimisation error, Chapter 3); a bad objective reaches the wrong target perfectly.

Reporting training risk; comparing models by accuracy when probabilities are what you use

Reporting the fitted model's training risk as its performance is the most common error in student reports: that number is optimistic by the proposition above. Fix: report a held-out number (Section 6). Second mistake: choosing between probabilistic models by accuracy and then deploying a threshold; accuracy is blind to calibration, cross-entropy is not. Third, a hand-computation slip: taking $-\log$ of the *largest* probability instead of the probability of the *true* class.

Decision loss versus training loss; zero–one versus cross-entropy

	Zero–one (a decision loss)	Cross-entropy (a training loss)
measures	whether the argmax was right	how much probability the true class got
in θ	piecewise constant, zero gradient	smooth, differentiable where $\hat{p} > 0$
minimiser in population	any rule with the Bayes argmax	the true conditional probabilities
sees confidence	no	yes: 0.9 beats 0.6, 0.1 is punished hard
can rank two models differently	yes: equal accuracy, different calibration	
role	judge the action; set with costs	fit the model; equals maximum likelihood

Key insight. ERM minimises what we can compute; its score is honest for a rule chosen in advance and flattering for the rule it chose, and cross-entropy is the training loss whose minimiser is the truth.

How the professor asks it

Q. *Exercises 2, Q4: two classifiers have the same accuracy on a validation set, but one has a much lower cross-entropy. Explain how this is possible, which one you would deploy with a cost-sensitive threshold, and why.*
A. Accuracy only counts whether the argmax was right; cross-entropy measures the probability placed on the true class. A model that says 0.51 on every correct case and 0.49 on every wrong

one can have the same accuracy as one that says 0.95 and 0.05, and far higher cross-entropy. Deploy the lower cross-entropy model: its expected cross-entropy is $H(Y \mid X)$ plus the KL from its probabilities to the truth, so lower means closer to the true posterior, which is what a threshold $C_{\mathrm{FP}}/(C_{\mathrm{FP}} + C_{\mathrm{FN}})$, a review band or a prior correction consume. Accuracy would not have seen the difference.

Q. *Derive the normal equations for a line fitted by least squares, and explain what the condition* $\partial \widehat{R}/\partial b = 0$ *says about the residuals.*
A. $\widehat{R}(w,b) = \frac{1}{N}\sum_i (y^{(i)} - wx^{(i)} - b)^2$. $\partial \widehat{R}/\partial b = -\frac{2}{N}\sum_i (y^{(i)} - wx^{(i)} - b) = 0$ gives $\sum y = w\sum x + Nb$; $\partial \widehat{R}/\partial w = -\frac{2}{N}\sum_i x^{(i)}(y^{(i)} - wx^{(i)} - b) = 0$ gives $\sum xy = w\sum x^2 + b\sum x$. The first equation says the residuals $y^{(i)} - \hat{y}^{(i)}$ sum to zero: the fitted line passes through the centroid $(\bar{x}, \bar{y})$. On $(0,1),(1,3),(2,2),(3,4)$ this gives $w = 0.8$, $b = 1.3$, residuals $-0.3, 0.9, -0.9, 0.3$, which do sum to zero.

Q. *Why can the training risk never increase when a polynomial's degree is raised, and why does that make it useless for choosing the degree?*
A. Degree-p polynomials are a subset of degree-$(p+1)$ polynomials (set the new coefficient to 0), so the minimum over the larger family is at most the minimum over the smaller. Pushed far enough, degree $N-1$ passes through all N points and the risk is 0. The training curve is therefore monotone in the degree and reaches its minimum at the most flexible family regardless of how that family predicts new points; a criterion that always answers "more" decides nothing. The degree must be chosen on validation data, where the curve is U-shaped.

Q. *A model assigns probability* 0.8 *to class* 1 *and the true label is* 0. *Compute the binary cross-entropy and explain why it is larger than the loss of the same model on an example with true label* 1.
A. $-\log(1-0.8) = -\ln 0.2 = 1.609$ nats, against $-\ln 0.8 = 0.223$ when the label is 1. The loss is minus the log of the probability given to what happened; a confident probability on the wrong class leaves 0.2 for the truth, and $-\ln$ grows without bound as that probability shrinks. Confident mistakes are the expensive ones, which is what makes the loss push probabilities towards calibration.

6 Evaluating a learned model: train, validation, test

Training, validation and test sets

The data are partitioned into three subsets by *role*, not by size:

Subset	Role	What it is allowed to influence
training $\mathcal{D}_{\mathrm{train}}$	learn the parameters θ through ℓ_{train}	θ only
validation $\mathcal{D}_{\mathrm{val}}$	compare fitted configurations, select hyperparameters, the threshold	λ, k, degree, features, threshold
test $\mathcal{D}_{\mathrm{test}}$	estimate the final performance, once, after every choice is fixed	nothing; it is consulted, never obeyed

A subset is informative about new data only until it is used to make a choice; then it has become "training" in disguise. Consulting the test set repeatedly turns it into a second validation set and makes its estimate optimistic by the mechanism of Section 5.

Purpose. Give each of the three decisions (parameters, hyperparameters, the reported number) evidence that was not already used to make it.

Impact. The reported test number is honest exactly to the extent that nothing was selected on it.

Held-out risk of a fixed model is honest; the best of several is not

If the model f was chosen without using the M held-out rows, drawn i.i.d., then $\frac{1}{M}\sum_i \ell(y^{(i)}, f(x^{(i)}))$ has expectation $R(f)$: identical to the fixed-model argument of Section 5, with the held-out rows in place of the training rows. [not on your sheet: know it]The catch is the word *fixed*. Select, among candidates $h \in \mathcal{C}$, the one with the lowest validation risk: for every h_0, $\min_h \widehat{R}_{\text{val}}(\hat{f}_h) \le \widehat{R}_{\text{val}}(\hat{f}_{h_0})$ pointwise; take expectations and then the minimum over h_0:

$$\mathbb{E}\big[\min_h \widehat{R}_{\text{val}}(\hat{f}_h)\big] \le \min_h R(\hat{f}_h) \quad \text{[on your sheet]}$$

with strict inequality as soon as the estimates are noisy (the winner's curse). The minimum of several noisy numbers is pulled below the truth. The test set restores honesty because nothing was selected on it.

Hyperparameter selection by validation

For each candidate λ learn the parameters on the training set, then compare the fitted models on the validation set and keep the best:

$$\hat{\theta}_\lambda = \arg\min_\theta \widehat{R}_{\text{train}}(\theta;\lambda), \qquad \hat{\lambda} = \arg\min_\lambda \widehat{R}_{\text{val}}(\hat{\theta}_\lambda;\lambda) \quad \text{[on your sheet]}$$

Two nested "pick the best" operations on two different datasets. Suppose polynomial degrees give training risks $1.25, 0.45, 0.40, 0.10, 0.00$ for degrees $0, 1, 2, 3, 5$ and validation risks $1.30, 0.50, 0.62, 1.90, 8.40$: training keeps falling (nested families), validation is smallest at degree 1; keep the line. The validation criterion may differ from ℓ_{train} when the application values something else (cost-weighted error, recall at a false-alarm budget).

Purpose. Choose complexity on evidence the fit did not see.

Impact. The selected validation score is itself optimistic (winner's curse); it is a selection device, not the number to report.

K-fold cross-validation

A single validation split leaves fewer rows for fitting and makes the choice depend on one accidental split. Divide the development data (training plus validation) into K folds; for each candidate λ and each fold j, train on the other $K-1$ folds and score on fold j; average:

$$\widehat{R}_{\text{CV}}(\lambda) = \frac{1}{K}\sum_{j=1}^{K} \widehat{R}^{(j)}_{\text{val}}(\lambda), \qquad \hat{\lambda} \in \arg\min_\lambda \widehat{R}_{\text{CV}}(\lambda) \quad \text{[on your sheet]}$$

Every development row validates exactly once and trains $K-1$ times. After selecting $\hat{\lambda}$, refit on all development data; the test set stays untouched. $K = N$ is leave-one-out. Cost: about K fits per candidate.

Purpose. Use limited data efficiently and average out the luck of one split.

Impact. Reduces the variance of the selection; does not remove the winner's curse, does not produce a final model by itself, and does not replace the test set.

The K-fold protocol, step by step

Setup. N rows of development data, a candidate set $\mathcal{C}$ (say $k \in \{1, 3, 5, 9, 15\}$ for kNN), $K = 5$.

Held fixed. The fold assignment, drawn once, randomly for exchangeable rows, by group or by time otherwise (see the Experiments document).

What we manipulate. The candidate λ and the held-out fold j.

What we measure. For each (λ, j): fit every data-dependent step (scaler, imputer, feature selection, the model) on the four training folds only; apply them to fold j; record the validation loss $\widehat{R}^{(j)}_{\text{val}}(\lambda)$. That is $|\mathcal{C}| \times K = 25$ fits.

What the plot shows. A table of K scores per candidate and their mean; a plot of $\widehat{R}_{\text{CV}}$ against λ with the spread across folds as error bars. With $K = 3$ and fold errors $0.40, 0.20, 0.60$ for $k = 1$ (mean 0.40) and $0.30, 0.30, 0.30$ for $k = 3$ (mean 0.30): choose $k = 3$. Fold 2 alone would have chosen $k = 1$.

How to read it. Pick the λ with the lowest mean, refit it on all N rows, evaluate once on the test set, report that. The error bars tell you whether two candidates are really different; a 10-row fold cannot separate 12% from 18%.

Fraud detection, the whole pipeline

Underlying problem: transactions X, fraud indicator Y; decision loss $C_{\text{FP}} = 5$ per blocked legitimate payment, $C_{\text{FN}} =$ the amount. Bayes rule: block iff $p(x) > 5/(5 + \text{amount})$; unknown, since p is. Data: two years of labelled transactions, split by *time* into development (first 20 months) and test (last 4). Family: a probabilistic classifier with hyperparameters (representation, model size). Training: minimise cross-entropy on $\mathcal{D}_{\text{train}}$ for each configuration. Validation: choose the configuration *and* the threshold by cost-weighted error on $\mathcal{D}_{\text{val}}$ (not by cross-entropy: the application cares about money). Test: apply the chosen policy to the last four months once and report the total cost. Deployment: the threshold stays adjustable because the policy is separate from the probabilities.

The five ways a reported number becomes a lie

(1) Reporting the training risk of the fitted model. (2) Reporting the validation minimum after selecting on it: the best of 500 noisy numbers is strongly optimistic. (3) Consulting the test set during development, which makes it a validation set. (4) Fitting the scaler, imputer, vocabulary or feature selection on all rows before splitting: test statistics leak into the representation. (5) Splitting rows at random when rows are grouped (near-duplicate flows of one attack, several images of one patient) or time-ordered: the test set then contains copies of the training set. Fix: decide the split before any result, by group or time when needed; fit every data-dependent step inside the training portion; select configuration and threshold on validation; touch the test set once; if you go back and change something, you need a new test set.

Training versus validation versus test

	Training	Validation	Test
decides	θ	λ, k, features, threshold	nothing
its risk is	optimistic for $\hat{\theta}$	honest per candidate, optimistic for the winner	honest, if used once
reused?	every fit	every comparison	never
replaced by	nothing	K-fold CV when data are scarce	a fresh set if you go back

Key insight. A number is a performance estimate only if the thing it measures was fixed before the number was computed.

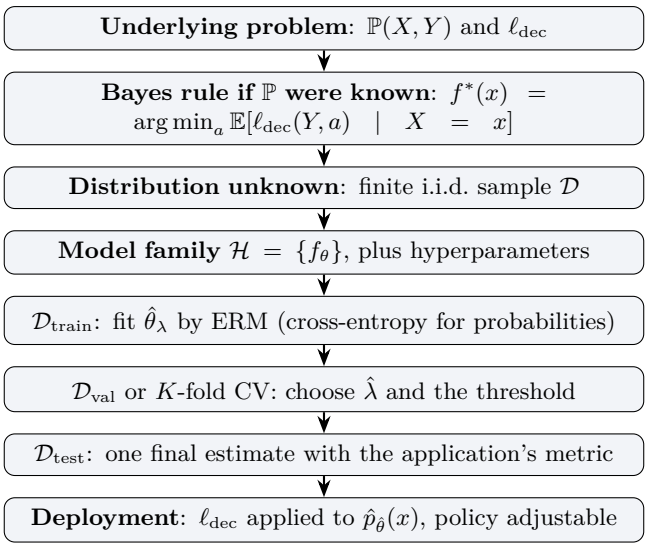
How the professor asks it

Q. *Exercises 2, Q10: a report standardises all rows before splitting, splits rows at random although each attack produced near-duplicate flows, tries* 500 *configurations and keeps the one with the best* test *accuracy, and thresholds at* 0.5 *although* $C_{\mathrm{FN}} = 100\, C_{\mathrm{FP}}$. *Give every reason the reported accuracy misleads and write the correct protocol.*
A. (1) Preprocessing leakage: test statistics shaped the inputs the model was trained on. (2) Duplicate leakage: the test set contains copies of training flows, so the score measures recognition of seen attacks. (3) Selection on the test set: the best of 500 noisy numbers is a validation score with a large winner's curse. (4) Accuracy at low prevalence hides missed attacks (the all-benign rule scores 0.99). (5) The Bayes threshold is $1/101 \approx 0.01$; at 0.5 every attack with posterior in $(0.01, 0.5)$ is missed. Protocol: deduplicate; split by episode and by time; fit scaling and selection inside each training fold; choose configuration and threshold on validation; evaluate once on the test set and report recall, precision and cost at the operating threshold.

Q. *Explain, using the optimism result, why the validation error of the chosen hyperparameter is not an estimate of the model's performance, and what is.*
A. The chosen $\hat{\lambda}$ minimises $\widehat{R}_{\mathrm{val}}$ over the candidates, so its validation score is the minimum of several noisy unbiased estimates, and $\mathbb{E}[\min_h \widehat{R}_{\mathrm{val}}(\hat{f}_h)] \leq \min_h R(\hat{f}_h)$: the minimum is pulled below the truth, more so with more candidates and noisier estimates. The validation set played the role of the training set and the candidates the role of Θ. The honest number is the test risk of the final model, computed once, after $\hat{\lambda}$ and the threshold are fixed.

Q. *Show with three folds that a single validation split can select the wrong k for kNN, and say what cross-validation does and does not fix.*
A. Fold errors $0.40, 0.20, 0.60$ for $k = 1$ (mean 0.40) and $0.30, 0.30, 0.30$ for $k = 3$ (mean 0.30). Validating on fold 2 alone picks $k = 1$; the average over folds picks $k = 3$. Cross-validation removes the dependence on one accidental split and uses every row for validation once. It does not remove the winner's curse (the selected 0.30 is still optimistic), it does not produce the final model (refit on all development data with $\hat{k}$), and it does not replace the untouched test set.

7 The learning problem in one picture



Experience is the finite sample; the hypothesis space says which rules may be considered; the loss measures the quality of outputs. Training risk guides parameter estimation, validation performance guides complexity, test performance estimates how the completed procedure behaves on new observations. Why a finite sample is evidence about future performance at all is the subject of Chapter 3.

How the professor asks it

Q. *Place the following in the picture and say which subset each is allowed to touch: the cost ratio, the number of neighbours k, the feature scales, the decision threshold, the reported recall.*
A. Cost ratio: part of the underlying problem (ℓ_{dec}), fixed by the application, touches no data. k and the feature scales: hyperparameters, chosen on validation or by cross-validation, never on training error (always $k = 1$) nor on the test set. Threshold: a policy on top of the probabilities, derived from the costs if the probabilities are calibrated, otherwise tuned on validation. Reported recall: computed once on the test set after everything above is frozen.

The ten mistakes of this chapter

1. Coding categories as integers, or leaving features in raw units, so that distance measures the coding instead of the object.
2. Predicting the most likely value under every loss; the mean, median, quantile and mode are answers to different losses.
3. Inverting the cost threshold: it is $C_{\text{FP}}/(C_{\text{FP}}+C_{\text{FN}})$; expensive misses give a *small* threshold and more alarms.
4. Treating the Bayes error as a learner's failure; it is the representation's, and the fix is a richer x.
5. Choosing k or the scales on training error (always $k = 1$) or on the query or test set.
6. Confusing the true distribution with the sample, and assuming i.i.d. for grouped or time-ordered rows.
7. Reporting the training risk of the fitted model; it is optimistic by construction.
8. Choosing complexity by training risk, which can only fall with flexibility, all the way to zero.
9. Comparing probabilistic models by accuracy and then deploying a threshold; accuracy is blind to calibration, cross-entropy is not.
10. Selecting on the test set, fitting the scaler before the split, or reusing the validation minimum as the reported number.

Chapter 2 in ten lines

1. Object $\to$ chosen features $\to$ vector in $\mathbb{R}^d$; both arrows are human choices; one-hot code categories; the representation bounds everything.
2. Uncertainty after seeing x is $\mathbb{P}(Y \mid X = x)$; the loss describes consequences; the Bayes rule minimises the conditional risk, pointwise and therefore globally.
3. Squared loss selects the conditional mean (add and subtract the mean, the cross term dies), absolute loss a median, asymmetric loss the τ-quantile with $\tau = c_u/(c_u + c_o)$.
4. Zero–one loss selects the most probable class and leaves the Bayes error; unequal costs give the threshold $C_{\text{FP}}/(C_{\text{FP}} + C_{\text{FN}})$; a fixed-cost review action adds a band $(C_{\text{R}}/C_{\text{FN}},\ 1 - C_{\text{R}}/C_{\text{FP}})$ iff $C_{\text{R}} < C_{\text{FP}}C_{\text{FN}}/(C_{\text{FP}} + C_{\text{FN}})$.
5. The sample is N i.i.d. rows from the unknown distribution; kNN plugs a local average into the Bayes rule; k, the distance and the scales are hyperparameters; capacity rises as k falls.
6. The curse: a point is within radius 0.1 with probability 10^{-d}, so $\mathbb{P}(\text{some neighbour}) = 1-(1-10^{-d})^n$ collapses at $d = 4$ for $n = 1000$; local methods need shared structure.
7. A parameterised family $\mathcal{H} = \{f_\theta\}$ shares structure across inputs; parameters are learned, hyperparameters chosen; the procedure decides which is which.

8. ERM minimises the empirical risk; honest for a fixed model, optimistic after fitting ($\mathbb{E}[\widehat{R}(\hat{\theta})] \leq \min R \leq \mathbb{E}[R(\hat{\theta})]$); larger families only lower training risk, down to zero at interpolation.
9. Cross-entropy $-\log \hat{p}_{\theta,y}(x)$ is smooth, equals negative log-likelihood, and its expectation is $H(Y \mid X) + \mathbb{E}[\mathrm{KL}]$, minimised by the true probabilities; zero–one is a staircase and cannot be trained on.
10. Training data learn θ, validation data (or K-fold CV) choose λ and the threshold, the test set gives one honest number; the best of several validation scores is optimistic; preprocessing fitted on all rows, duplicates across subsets and selection on the test set all inflate the estimate.

Coverage of the textbook chapter

Textbook item (Chapter 2)	Where in this document
Symbol table; input space, feature vector, one-hot; "why not $1, 2, 3$"; three email descriptions; gotchas on codes and units	Section 1
Conditional distribution; decision loss; Bayes rule and conditional risk; pointwise = global (proof)	Section 2, first two boxes
Squared loss → mean (proof); absolute → median (proof); asymmetric → quantile (proof); delivery example; loss table	Section 2, regression boxes and versus table
Zero–one → argmax, Bayes error; cost threshold (proof); detection, fraud, court, screening examples; triage cost table; technique; classics; gotchas	Section 2, classification boxes, example, warning, drill
Reject / review band (Essentials, Tutorial 1 Q2, on the sheet)	Section 2, review box
Training dataset, i.i.d.; true distribution vs sample; kNN definition; kNN by hand; Voronoi; distances; scaling example; standardise on training rows	Section 3
Grid count; probability of a neighbour (proof, table); computation; kNN technique; classics ($k = 6$, expected count, 1-NN error)	Section 3, math box, example, drill
Parameterised model, hypothesis space, inductive bias; parameters vs form; nested spaces; simplex, softmax; parameters vs hyperparameters table; lazy gotcha	Section 4
Two losses; population and empirical risk, ERM; honest (proof); optimistic (proof); nested (proof); interpolation (proof); least squares by hand; staircase example; cross-entropy definition and table; $H + \mathrm{KL}$ theorem (proof); MLE (proof); objective vs algorithm; technique; classics	Section 5
Three subsets; held-out honest (proof); selection optimism; hyperparameter selection with the degree table; K-fold CV with the fold table; splitting technique; gotchas	Section 6
The picture; fraud pipeline; notes on the slides (median not unique, centre-of-ball caveat, softmax not examinable from this lecture)	Section 7, Section 6 example, Sections 2–3